OCR H446 / PAPER 2 / UNIT 2.2.2

Computational methods

CHAPTER CONTENTS

Select a topic to jump to its original worked pages.

09 Computational solvability	2
10 Problem decomposition	6
11 Divide and conquer	10
12 Abstraction	14
13 Backtracking	18
14 Data mining	22
15 Heuristics	26
16 Performance modelling	30
17 Pipelining	34
18 Visualisation	38

2.2.2(a,b) / SEE IT / CONCEPT AND VISUAL

Turn a vague request into a computational problem

A computationally solvable problem needs defined inputs, a specified output, workable rules and finite executable steps within practical resource limits.

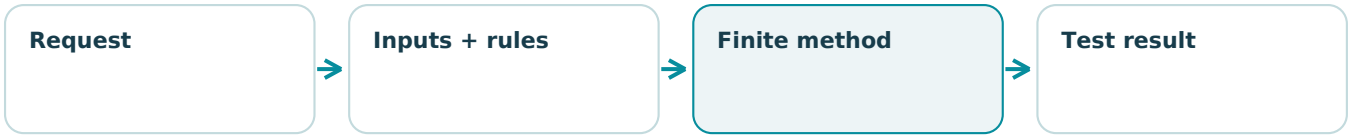
RECOGNISE
State the actual problem.

INPUT
Specify valid data and constraints.

OUTPUT
Define an observable result.

FEASIBILITY
Can a procedure meet resource limits?

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

"Make traffic better" is not a full computational specification. "Find a route with minimum recorded journey time on a fixed road graph" is testable and constrained.

WORDS THAT EARN MARKS

Recognition	Identify problem	**Constraint**	Required limit
Feasible	Practical to solve	**Computable**	Algorithmic solution

EXAM TRAP

Not every human judgement is a well-defined algorithmic problem until goals, rules and acceptable trade-offs are stated.

2.2.2(a,b) / WORKED EXAMPLE / STEP BY STEP

Frame a bus-routing request before choosing an algorithm

A school wants a route from Depot D to School S on a road network. Each road has a non-negative estimated travel time.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Goal	**Minimise predicted travel time**	Convert vague wish into a target.
2 / Inputs	**Road graph, D, S, edge times**	Identify needed data.
3 / Constraints	**Only allowed roads; times >= 0**	Specify valid solution space.
4 / Output	**A route plus predicted minutes**	Define verifiable result.
5 / Method	**Shortest-path search**	Problem has graph structure.
6 / Test	**A small known network with expected route**	Check solution before real deployment.

TRANSFER IT TO AN EXAM ANSWER

Dynamic traffic conditions require updated weights or a stated model limitation. Practical feasibility considers memory, execution time and quality of data.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(a,b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Give three features of a well-specified computational problem.

Q2 / 5 MARKS

Explain how to turn "find a good route" into an algorithmic problem, specifying input, output and success criterion.

Q3 / 6 MARKS

Evaluate whether a computer can choose a "fair" timetable for all pupils without further information; describe what must be defined first.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Explicit input [1]; precise output or success criterion [1]; usable rules/constraints and finite processing [1].

Q2 / 5

Define the desired criterion, e.g. minimum minutes [1]; road graph [1]; start and destination [1]; legal-route constraints [1]; output route and total minutes [1].

Q3 / 6

"Fair" is ambiguous [1]; identify affected groups [1]; define measurable priorities [1]; specify hard constraints [1]; choose objective/trade-offs [1]; evaluate with stakeholders and real-world cases [1].

COMMON MISCONCEPTION

Not every human judgement is a well-defined algorithmic problem until goals, rules and acceptable trade-offs are stated.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(c) / SEE IT / CONCEPT AND VISUAL

Name genuine sub-problems and their interfaces

Decomposition divides a larger problem into smaller sub-problems that can be specified, solved and tested before integration.

TOP LEVEL

One clear problem.

SUB-PROBLEM

A manageable independent responsibility.

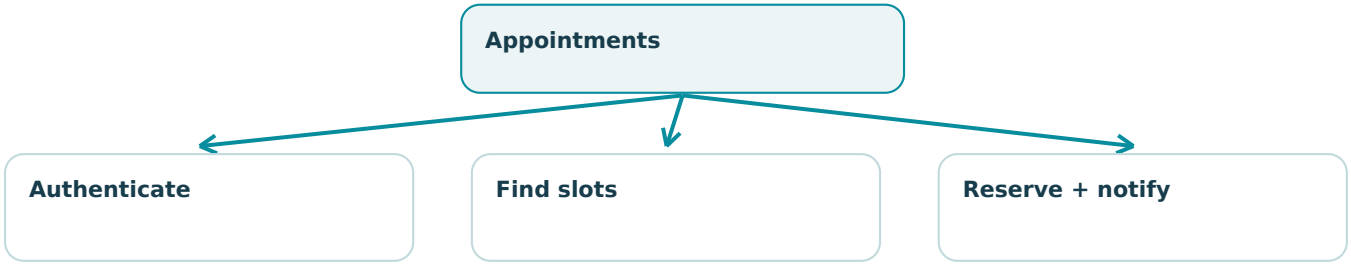
INTERFACE

What data enters and leaves.

INTEGRATION

Combine tested parts in the right order.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

A decomposition diagram should show meaningful tasks, not vague phrases like "do first part". Keep data flow visible between sub-problems.

WORDS THAT EARN MARKS

- **Decomposition** — Smaller sub-problems
- **Module** — Coherent component
- **Interface** — Input/output contract
- **Integration** — Join components

EXAM TRAP

Decomposition is not the same as abstraction: decomposition separates responsibilities; abstraction removes irrelevant detail.

2.2.2(c) / WORKED EXAMPLE / STEP BY STEP

Break down a parents-evening appointment system

A system receives pupil/parent requests, offers available teacher slots, saves a booking and sends a confirmation.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Identify	**Manage appointments**	Top-level problem.
2 / Subproblem	**Authenticate parent**	Check eligibility.
3 / Subproblem	**Find available slots**	Read teacher and slot data.
4 / Subproblem	**Reserve chosen slot**	Make a consistent state update.
5 / Subproblem	**Send confirmation**	Use booking ID as input.
6 / Integrate	**Authenticate -> choose -> reserve -> notify**	Define dependency between outputs.

TRANSFER IT TO AN EXAM ANSWER

Each sub-problem can have a separate test: for example, an already-booked slot must not be offered. Integration tests then check complete workflows.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(c) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define decomposition and give two benefits.

Q2 / 5 MARKS

Construct four sub-problems for the appointment system, with one input/output pair.

Q3 / 6 MARKS

Explain how interfaces and tests would prevent a double-booked teacher slot when integrating modules.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Break into smaller sub-problems [1]; easier isolated testing [1]; easier teamwork/reuse/maintenance [1].

Q2 / 5

Authenticate parent [1]; list availability [1]; reserve slot [1]; confirm booking [1]; interface example: teacher and date in, free slots out [1].

Q3 / 6

Specify one reservation API [1]; validate desired slot [1]; check availability at commit [1]; atomically reserve [1]; unit test clashes [1]; integrate two simultaneous booking requests [1].

COMMON MISCONCEPTION

Decomposition is not the same as abstraction: decomposition separates responsibilities; abstraction removes irrelevant detail.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(d) / SEE IT / CONCEPT AND VISUAL

Split, solve recursively and combine

Divide and conquer recursively splits a problem into smaller related instances, solves them, then combines their results.

DIVIDE

Split input into parts.

CONQUER

Solve each part; recurse if needed.

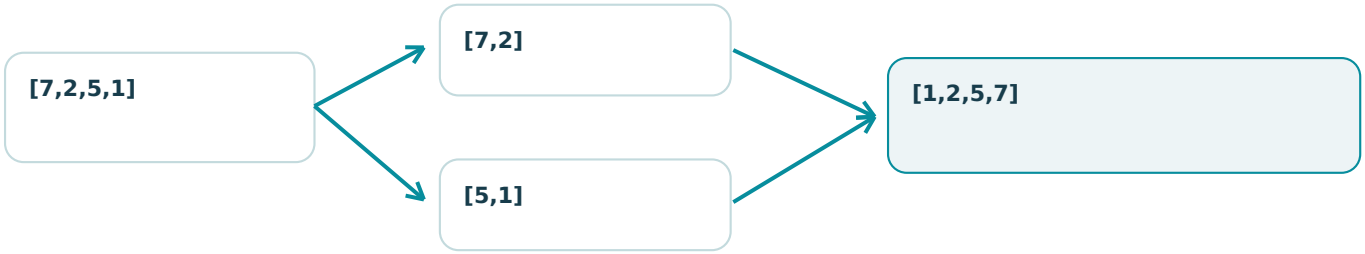
COMBINE

Join partial results.

BASE CASE

Stop when small enough.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Merge sort is a concrete divide-and-conquer example: the final result depends on combining two already-sorted halves.

WORDS THAT EARN MARKS

- **Divide** Split problem
- **Conquer** Solve parts
- **Combine** Merge answers
- **Recurrence** Repeated subproblems

EXAM TRAP

Divide and conquer does not mean "split once and stop"; explain what is done to each sub-problem and how answers combine.

2.2.2(d) / WORKED EXAMPLE / STEP BY STEP

Follow one merge-sort split and merge completely

Input array is [7, 2, 5, 1]. The merge operation repeatedly selects the smallest front item of two sorted lists.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Divide	**[7,2] and [5,1]**	Split array in halves.
2 / Divide	**[7],[2],[5],[1]**	Single values are sorted base cases.
3 / Merge left	**[2,7]**	Compare 7 and 2.
4 / Merge right	**[1,5]**	Compare 5 and 1.
5 / Merge	**1, then 2, then 5, then 7**	Select lower available front each time.
6 / Result	**[1,2,5,7]**	All elements retained and ordered.

TRANSFER IT TO AN EXAM ANSWER

Divide and conquer is an algorithm design method, not just dividing work between programmers. Merge sort typically uses O(n log n) time and O(n) extra space for arrays.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

State divide, conquer and combine.

Q2 / 5 MARKS

Show every split and merge for [7,2,5,1].

Q3 / 6 MARKS

Compare using divide and conquer in merge sort with a simple repeated adjacent-swap approach, commenting on time and space.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Divide the problem [1]; solve parts recursively/base cases [1]; combine results [1].

Q2 / 5

Split [7,2] / [5,1] [1]; four singleton lists [1]; merge [2,7] [1]; merge [1,5] [1]; final [1,2,5,7] [1].

Q3 / 6

Merge sort splits recursively [1], combines sorted lists [1], worst-case time O(n log n) [1]; bubble sort has O(n^2) worst-case [1]; merge sort requires auxiliary storage in usual array version [1]; choice depends on input size and memory [1].

COMMON MISCONCEPTION

Divide and conquer does not mean "split once and stop"; explain what is done to each sub-problem and how answers combine.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(e) / SEE IT / CONCEPT AND VISUAL

Simplify a real situation without losing correctness

In computational methods, abstraction identifies the features needed to solve the stated problem and deliberately ignores unrelated detail.

PURPOSE

Specify the exact question.

KEEP

Variables that affect the decision.

IGNORE

Details irrelevant to the question.

VALIDATE

Check the assumptions and exceptions.

HOW THE PARTS FIT TOGETHER



Ignore cover colour; retain due date and return state.

FROM THE DIAGRAM TO EXAM LANGUAGE

A model for overdue notices needs due dates, return state and borrower identity; it does not need book-cover artwork.

WORDS THAT EARN MARKS

Abstraction	Remove irrelevant	**Model**	Simplified system
Assumption	Rule treated as true	**Validation**	Check model

EXAM TRAP

A model is not automatically valid forever; new requirements can make an omitted detail relevant.

2.2.2(e) / WORKED EXAMPLE / STEP BY STEP

Design the smallest useful overdue-loan model

Library policy: notify a borrower if a book is unreturned and today is later than its due date. Today is day 15; loan due day 12 is unreturned.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Purpose	**Decide whether to notify**	One specified outcome.
2 / Keep	**borrowerID, dueDay, returned**	All can affect output.
3 / Ignore	**coverColour, shelfDecoration**	Do not affect notification.
4 / Rule	**NOT returned AND today>dueDay**	Boolean predicate.
5 / Evaluate	**TRUE AND 15>12 => TRUE**	This loan is overdue.
6 / Output	**Notify borrower**	Relevant result from the model.

TRANSFER IT TO AN EXAM ANSWER

If loans can be renewed or due dates shifted on closure days, those real-world features may need to be restored to the model.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(e) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define abstraction and name two relevant loan fields.

Q2 / 5 MARKS

Identify two excluded details and explain why the sample loan triggers a notice.

Q3 / 6 MARKS

Evaluate how the model should change if pupils may renew loans online or return them during closure periods.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(e) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Abstraction omits irrelevant detail [1]; due date [1]; return status or borrower identifier [1].

Q2 / 5

Book-cover colour and shelf decoration irrelevant [2]; not returned [1]; day 15 later than day 12 [1]; therefore notify [1].

Q3 / 6

Keep renewal status or revised due date [1]; include actual return timestamp [1]; define closure-day policy [1]; update notification predicate [1]; test same-day/renewed/closure boundaries [1]; validate against library policy [1].

COMMON MISCONCEPTION

A model is not automatically valid forever; new requirements can make an omitted detail relevant.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Try a choice; undo it if it blocks the goal

Backtracking explores candidate choices systematically. When a partial solution violates constraints or reaches a dead end, undo the choice and try another.

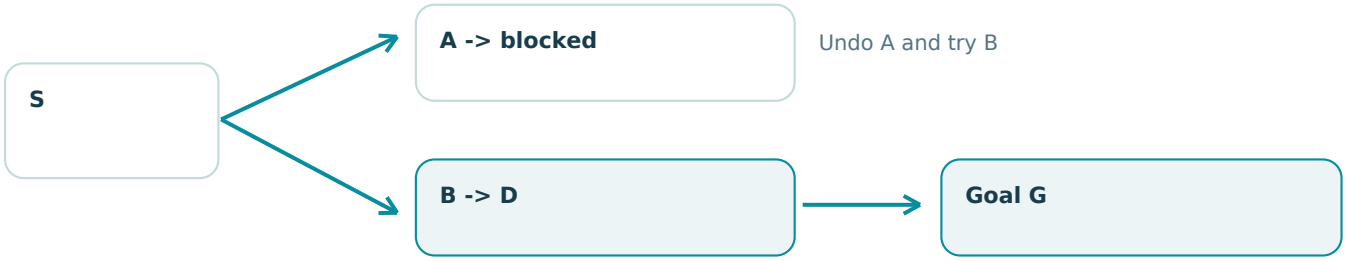
CHOOSE
Add one possible step.

TEST
Does it respect constraints?

BACKTRACK
Undo a failed choice.

STOP
Goal found, or choices exhausted.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

The search must record the current path or state. Undoing an invalid candidate is not randomly restarting from zero.

WORDS THAT EARN MARKS

Backtracking	Undo decisions	**Search tree**	Possible choices
Constraint	Rule to satisfy	**Dead end**	No valid next step

EXAM TRAP

Backtracking is an exhaustive-search strategy with pruning, not a guarantee of efficient performance on large inputs.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Find a legal path through a tiny maze

From Start S choose A or B. A leads to C, which is blocked. B leads to D and then Goal G. Depth-first order tries A first.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Start	**Path [S]**	No decision made yet.
2 / Choose A	**[S,A]**	First branch is explored.
3 / Try C	**[S,A,C] => blocked**	Constraint violation discovered.
4 / Undo	**Return to [S]**	Backtrack past failed A branch.
5 / Choose B	**[S,B,D]**	Second branch remains available.
6 / Goal	**[S,B,D,G]**	Valid path found.

TRANSFER IT TO AN EXAM ANSWER

If no branch leads to a goal, the algorithm must report failure rather than pretending that its last partial path is complete.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define backtracking, a dead end and a search tree.

Q2 / 5 MARKS

Trace the first failed choice and successful path in the maze.

Q3 / 6 MARKS

Describe a backtracking algorithm for assigning pupils to limited places in clubs; give pruning conditions and one drawback.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Backtracking undoes choices [1]; dead end violates requirements/has no options [1]; search tree represents alternative choices [1].

Q2 / 5

Try S-A [1], then C [1], find it blocked [1], undo A [1], choose S-B-D-G [1].

Q3 / 6

Assign next pupil [1]; try club with capacity and preference rules [1]; recurse on partial assignment [1]; undo when no valid extension [1]; prune capacity/eligibility violations early [1]; worst-case combinations may grow exponentially [1].

COMMON MISCONCEPTION

Backtracking is an exhaustive-search strategy with pruning, not a guarantee of efficient performance on large inputs.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Find patterns, then test whether they are useful

Data mining searches large data sets for trends, associations and anomalies. A discovered correlation does not by itself establish a cause.

COLLECT
Gather relevant, lawful data.

CLEAN
Handle errors and missing records.

DISCOVER
Identify repeated patterns.

VALIDATE
Check patterns on separate data.

HOW THE PARTS FIT TOGETHER



60% and 20% are observed group rates, not a causal statement.

FROM THE DIAGRAM TO EXAM LANGUAGE

A group with lower attendance and lower scores shows an association, but unseen factors may influence both. A model should be tested, not simply trusted.

WORDS THAT EARN MARKS

Data mining	Pattern discovery	**Correlation**	Association
Outlier	Unusual case	**Bias**	Systematic skew

EXAM TRAP

An observed correlation is not a causal explanation or permission to make automated high-stakes decisions.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Interpret a small aggregated pattern responsibly

Among 100 pupils, 20 have attendance under 90%; 12 of those 20 score below a target. Of 80 pupils at 90% or above, 16 score below target.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Group A	**12 / 20 = 60%**	Low-attendance pupils below target.
2 / Group B	**16 / 80 = 20%**	Other pupils below target.
3 / Compare	**Difference = 40 percentage points**	An association appears in these data.
4 / Avoid	**Do NOT claim low attendance caused result**	Other factors may matter.
5 / Validate	**Compare a new cohort / held-out data**	Check whether pattern generalises.
6 / Safeguard	**Limit access and inspect bias**	Avoid unfair decisions about individuals.

TRANSFER IT TO AN EXAM ANSWER

The percentages are group observations; they do not predict an individual pupil with certainty. Any support decision needs appropriate safeguards.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define data mining, correlation and an outlier.

Q2 / 5 MARKS

Calculate both rates and explain why the pattern is not proof of causation.

Q3 / 6 MARKS

Outline a defensible data-mining workflow to study attendance and outcomes, with privacy, representativeness and evaluation.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Data mining seeks patterns [1]; correlation means association [1]; outlier is unusual observation [1].

Q2 / 5

Low group 12/20=60% [1]; high group 16/80=20% [1]; difference 40 percentage points [1]; association detected [1]; other causes and confounders exist [1].

Q3 / 6

Specify legitimate question [1]; use minimised, appropriately protected data [1]; clean missing/outlier errors [1]; explore patterns [1]; validate on independent sample [1]; check representation and consequences before acting [1].

COMMON MISCONCEPTION

An observed correlation is not a causal explanation or permission to make automated high-stakes decisions.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Find a useful answer without exhaustive search

A heuristic is a rule-of-thumb that directs search or produces a quick approximate solution. It may improve speed without guaranteeing the best result.

RULE

A fast decision strategy.

ADVANTAGE

Reduce search effort.

COST

Can miss optimum.

EVALUATE

Compare quality and runtime.

HOW THE PARTS FIT TOGETHER

A -> B -> D

2+10 = 12

A -> C -> D

3+4 = 7

Greedy is worse

FROM THE DIAGRAM TO EXAM LANGUAGE

Choosing the next nearest stop is a heuristic: it uses local information. The globally best route can require a locally worse choice.

WORDS THAT EARN MARKS

- **Heuristic** Rule of thumb
- **Greedy** Local choice
- **Optimal** Best possible
- **Approximate** May not be best

EXAM TRAP

"Heuristic" is not another name for an exact algorithm. State the trade-off between speed and guaranteed optimality.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Expose a greedy-choice trap with a tiny graph

The goal is travel from A to D. Direct edges: A-B=2, A-C=3, B-D=10, C-D=4. A greedy nearest-first choice prefers B.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Options	**A-B 2 versus A-C 3**	Greedy rule prefers smaller immediate cost.
2 / Choose B	**Cost so far 2**	Local choice committed.
3 / Continue	**B-D 10**	Only listed onward option.
4 / Greedy total	**2 + 10 = 12**	Candidate route A-B-D.
5 / Alternative	**A-C-D = 3 + 4 = 7**	Locally longer first edge is better.
6 / Conclusion	**Heuristic result is not optimal**	Compare complete path costs.

TRANSFER IT TO AN EXAM ANSWER

Use a provably suitable method such as Dijkstra on non-negative weighted graphs when guaranteed shortest path is required; do not confuse "nearest first" with Dijkstra.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define heuristic, greedy and optimal.

Q2 / 5 MARKS

Compare routes A-B-D and A-C-D and explain the poor greedy choice.

Q3 / 6 MARKS

Evaluate a heuristic for scheduling homework tasks under a tight processing-time limit, including how to judge solution quality.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Heuristic guides search quickly [1]; greedy takes a locally preferred option [1]; optimal is best according to objective [1].

Q2 / 5

A-B-D=12 [1]; A-C-D=7 [1]; heuristic selects B because 2<3 [1]; total 12 is worse than 7 [1]; local decision not globally best [1].

Q3 / 6

Specify objective [1]; choose priority rule [1]; improve runtime compared with searching all schedules [1]; test on known optimal small cases [1]; measure quality gap [1]; accept only if result is adequate for context [1].

COMMON MISCONCEPTION

"Heuristic" is not another name for an exact algorithm. State the trade-off between speed and guaranteed optimality.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Predict work, then test the prediction

Performance modelling uses simplified mathematical or simulated representations to estimate resource use, response time or throughput before deployment.

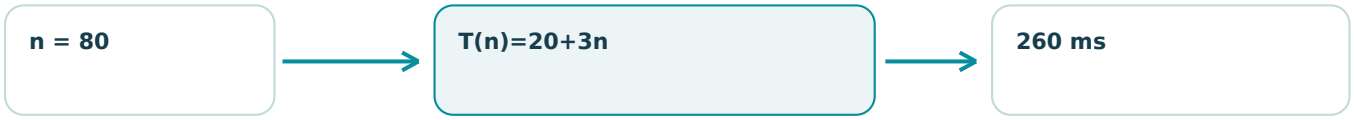
ASSUMPTION
Declare what is held constant.

MODEL
Predict time or capacity.

MEASURE
Collect observed results.

REVISE
Explain discrepancies.

HOW THE PARTS FIT TOGETHER



Measured runtime may differ because assumptions are simplified.

FROM THE DIAGRAM TO EXAM LANGUAGE

Use an explicitly assumed per-item cost to make a prediction; real systems add startup costs, caching, contention and I/O latency.

WORDS THAT EARN MARKS

Performance Time or resources

Throughput Completed per second

Latency Time per request

Bottleneck Limiting stage

EXAM TRAP

An algebraic model is a prediction based on assumptions; measurements are still required.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Build a small cost model for a serial batch

A program processes 80 identical records, taking an assumed 3 ms per record plus a fixed 20 ms setup cost. Assume no overlap.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Define	**T(n) = 20 + 3n ms**	Linear model with startup cost.
2 / Substitute	**n = 80**	Batch size is known.
3 / Compute	**T(80) = 20 + 240 = 260 ms**	Predicted completion time.
4 / Scale	**T(160) = 20 + 480 = 500 ms**	Doubling input does not double setup.
5 / Measure	**Observe actual timings**	Compare against prediction.
6 / Refine	**Account for cache or I/O variance**	Update assumptions if needed.

TRANSFER IT TO AN EXAM ANSWER

The model cannot justify a real-world guarantee without collecting evidence and checking whether per-item cost remains stable.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define latency, throughput and bottleneck.

Q2 / 5 MARKS

Use T(n)=20+3n to predict 80 and 160 records; explain one model assumption.

Q3 / 6 MARKS

Design a fair performance experiment comparing two solutions to the same problem, including repetition and data-set effects.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Latency is elapsed response time [1]; throughput is number completed per unit time [1]; bottleneck limits overall performance [1].

Q2 / 5

T(80)=260 ms [2]; T(160)=500 ms [2]; each record assumed to cost 3 ms regardless of size or order [1].

Q3 / 6

Use identical hardware/environment [1]; same representative inputs [1]; measure wall time and relevant memory [1]; repeat trials [1]; inspect mean/distribution and anomalies [1]; compare against model with caveats [1].

COMMON MISCONCEPTION

An algebraic model is a prediction based on assumptions; measurements are still required.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Overlap stages instead of waiting for each task

Pipelining divides work into stages so different tasks can occupy different stages at the same time. Throughput may improve after fill-up.

STAGES

Sequential operations for one item.

OVERLAP

Different items in different stages.

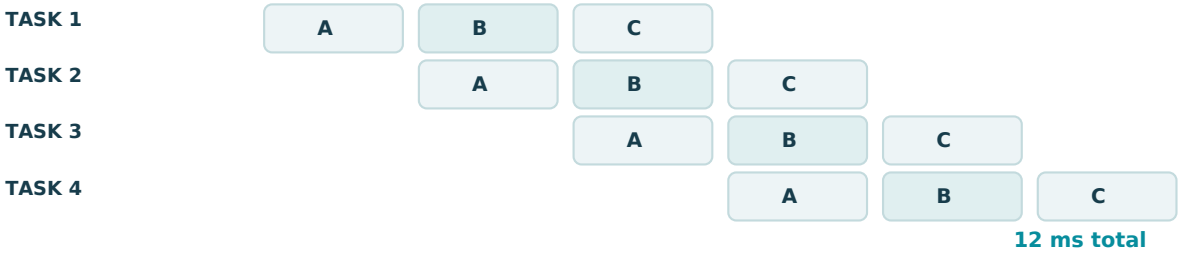
LATENCY

One item still traverses all stages.

THROUGHPUT

More items finish per time unit.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

An idealised three-stage pipeline makes four tasks complete faster as a batch, but does not make the latency of the first task one stage long.

WORDS THAT EARN MARKS

- **Pipeline** Overlapping stages
- **Fill** Start-up cycles
- **Latency** Time for one
- **Hazard** Prevents overlap

EXAM TRAP

Pipeline speed-up is a batch throughput improvement, not a guarantee that each single operation becomes that many times faster.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Compare serial and pipelined work with equal stages

Each task requires three 2 ms stages (A,B,C). Four independent tasks pass through the stages; assume no overhead or stalls.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Serial item	**2+2+2 = 6 ms**	One task traverses all stages.
2 / Serial batch	**4 x 6 = 24 ms**	No overlap.
3 / Pipeline fill	**First completion at 6 ms**	Latency of first task unchanged.
4 / Steady state	**Next finishes every 2 ms**	One completion per stage period.
5 / Four tasks	**(3+4-1) x 2 = 12 ms**	Ideal batch completion time.
6 / Compare	**24 ms versus 12 ms**	Batch throughput improves twofold here.

TRANSFER IT TO AN EXAM ANSWER

If one stage takes 5 ms, a simple fixed-stage pipeline cannot finish items faster than that bottleneck stage. Dependencies and hazards can cause stalls.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish pipeline latency and throughput; state one hazard.

Q2 / 5 MARKS

Calculate serial and pipelined completion times for four tasks with three 2 ms stages.

Q3 / 6 MARKS

Explain why a real pipeline might not achieve the ideal speed-up; apply dependency and bottleneck arguments.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Latency means one task completion time [1]; throughput means completions over time [1]; a hazard blocks ideal overlap [1].

Q2 / 5

Serial: 4 x 6=24 ms [2]; pipeline (3+4-1)x2=12 ms [2]; speed-up 24/12=2 [1].

Q3 / 6

Stage imbalance limits clock/period [1]; data dependency may stall [1]; branch or control hazard may flush work [1]; filling/draining adds cost [1]; communication/synchronisation has overhead [1]; so ideal throughput may not be achieved [1].

COMMON MISCONCEPTION

Pipeline speed-up is a batch throughput improvement, not a guarantee that each single operation becomes that many times faster.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Choose a picture that reveals the computation

Visualisation turns data or process state into diagrams or charts that reveal trends, bottlenecks or relationships. Choosing the wrong scale or chart misleads.

TABLE

Exact values are visible.

BAR CHART

Compare categories.

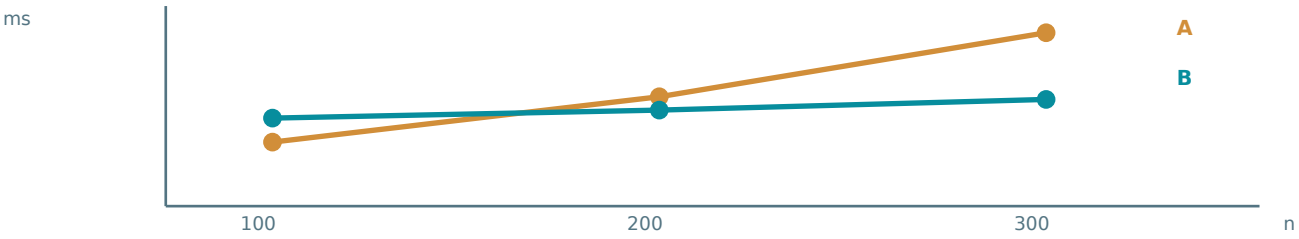
LINE GRAPH

Show change over time.

GRAPH NETWORK

Show connections or routes.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Choose an appropriate encoding for the question: a bar chart compares algorithms at a fixed n; a line chart shows how execution time changes as n increases.

WORDS THAT EARN MARKS

Visualisation	Graphical display	**Axis**	Scaled dimension
Trend	Overall pattern	**Misleading scale**	Distorted reading

EXAM TRAP

A neat graphic is not necessarily an accurate representation; axes, units, sample size and context affect validity.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Visualise a performance comparison without hiding labels

Algorithm A times (ms) for n=100,200,300: [8,16,24]. Algorithm B: [12,13,14]. Draw n on the horizontal axis and runtime on vertical axis.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Data	**n=100,200,300**	Input size is independent variable.
2 / A	**8,16,24 ms**	A grows by 8 ms per +100 records.
3 / B	**12,13,14 ms**	B grows by 1 ms per +100 records.
4 / Plot	**Two labelled series**	Use same axes for fair comparison.
5 / Compare	**At n=100, A faster (8<12)**	A starts with smaller observed time.
6 / Compare	**At n=300, B faster (14<24)**	Performance ranking changes with n.

TRANSFER IT TO AN EXAM ANSWER

Plot measured values only and explain what the chart does not prove: three measurements do not establish a full asymptotic complexity class.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Name a suitable visual for time series, categories and graph connections.

Q2 / 5 MARKS

Interpret the A/B timings at n=100 and n=300. Identify the axes and units.

Q3 / 6 MARKS

Critique a graph comparing algorithm times that truncates the y-axis, lacks units and mixes different hardware results.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Line chart for time series [1]; bar for categories [1]; network diagram for connections [1].

Q2 / 5

Horizontal input size n [1]; vertical time in ms [1]; n=100 A faster 8<12 [1]; n=300 B faster 14<24 [1]; ranking changes as input grows [1].

Q3 / 6

Truncated scale can exaggerate relative change [1]; missing units prevent interpretation [1]; different hardware confounds comparison [1]; use consistent apparatus [1]; label both axes and legend [1]; show honest scale/data plus methodological notes [1].

COMMON MISCONCEPTION

A neat graphic is not necessarily an accurate representation; axes, units, sample size and context affect validity.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/